

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 2

Industry Problem Solving Task

COURSE NAME: Deep Learning

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO 1: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. (BL-6)

Assessment Tool Weightage: 40%

QUESTIONS:

Total Marks:25

Q1. Transfer Learning

A healthcare startup aims to develop an AI-based system for detecting skin cancer from dermoscopic images. Since only a limited number of labeled images are available, design a transfer learning strategy using a pre-trained CNN model. Justify your choice of model, identify which layers should be frozen or fine-tuned, and explain how your design improves classification performance.

Q2. DenseNet and PixelNet

A smart city project requires an automated road-scene analysis system that accurately identifies roads, pedestrians, vehicles, and traffic signs at the pixel level. Design a deep learning architecture using DenseNet and PixelNet to solve this problem. Explain how the proposed architecture improves feature reuse, segmentation accuracy, and computational efficiency.

Q3. Bidirectional RNN and Encoder–Decoder

A multinational customer service company wants to build a multilingual chatbot capable of translating customer queries while preserving contextual meaning. Design an Encoder–Decoder sequence-to-sequence model using Bidirectional RNNs. Illustrate the architecture and justify how bidirectional processing improves translation accuracy.

Q4. BPTT and LSTM

A financial analytics company is developing a model to forecast stock prices using historical market data. Create an LSTM-based prediction system and explain how Backpropagation Through Time (BPTT) is used to train the network. Justify why LSTM is more suitable than a traditional RNN for this application.

Q5. Integrated Deep Learning Solution

A media streaming company plans to automatically generate captions for uploaded videos to improve accessibility. Design an integrated deep learning solution that combines Transfer Learning for visual feature extraction with an LSTM-based Encoder–Decoder architecture for

caption generation. Explain the role of each component and justify how the complete system produces accurate captions.

Code of Conduct:

I S.Mahammed Nayeem/192425294 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S. Mahammed Nayeem

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

Q1. Transfer Learning for Skin Cancer Detection

Proposed Model: ResNet50

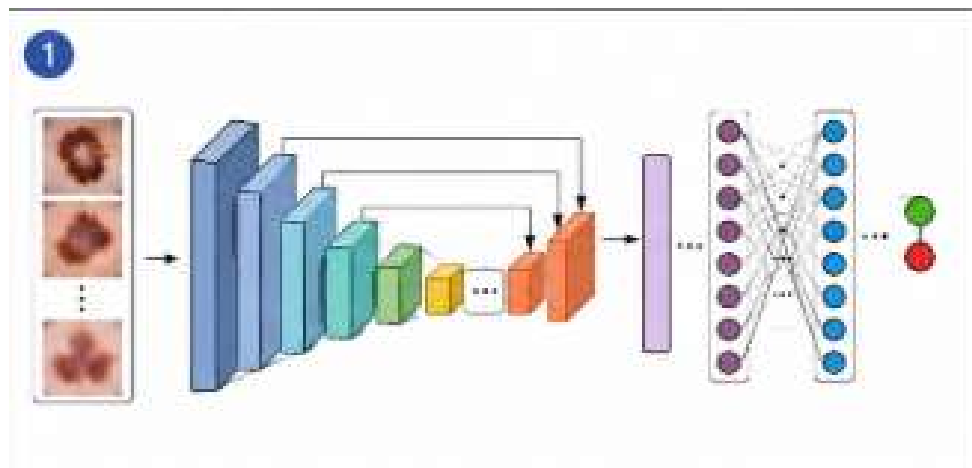
A healthcare startup can use **Transfer Learning with a pre-trained ResNet50 CNN** for skin cancer detection. Since only a limited number of labeled dermoscopic images are available, training a deep CNN from scratch may lead to overfitting. A pre-trained model provides useful visual features learned from a large dataset.

Architecture

Dermoscopic Image → ResNet50 Feature Extractor → Global Average Pooling → Fully Connected Layer → Dropout → Output Layer → Skin Cancer Class

Why ResNet50?

- ResNet50 is a deep CNN with residual connections.
- It can learn complex image features effectively.
- Pre-trained ImageNet weights provide useful low-level features such as edges, textures, shapes, and patterns.
- Residual connections improve gradient flow during training.
- It requires less training data compared with training a complete CNN from scratch.
- It provides good classification performance with comparatively reasonable computational requirements.



Layers to Freeze

Initially, the early layers of ResNet50 should be frozen because they learn general features.

- Freeze the initial convolutional layers.
- Freeze most of the middle layers.
- Remove the original ImageNet classification layer.
- Add a new classification head suitable for skin cancer classes.

The new layers are trained first.

Layers to Fine-Tune

After training the new classification layer:

- Unfreeze the last few convolutional blocks.
- Fine-tune these layers using a small learning rate.
- Keep the early layers frozen because their features are generally useful.
- Deeper layers can adapt to skin-lesion-specific characteristics.

Data Augmentation

Because the dataset is small, augmentation can be used:

- Rotation
- Horizontal flipping
- Vertical flipping
- Zooming
- Random cropping
- Brightness adjustment
- Slight scaling

Training Process

1. Collect labeled dermoscopic images.
2. Resize images to 224×224 .
3. Normalize pixel values.
4. Apply data augmentation.
5. Load pre-trained ResNet50.
6. Remove the original classification layer.
7. Add Global Average Pooling and Dense layers.
8. Freeze early layers.
9. Train the new classification head.
10. Unfreeze selected deeper layers.
11. Fine-tune using a low learning rate.
12. Evaluate using accuracy, precision, recall, F1-score, and confusion matrix.

Performance Improvement

Transfer learning improves classification because the CNN already contains useful visual representations. Only the task-specific features need to be learned from the limited medical dataset.

It provides:

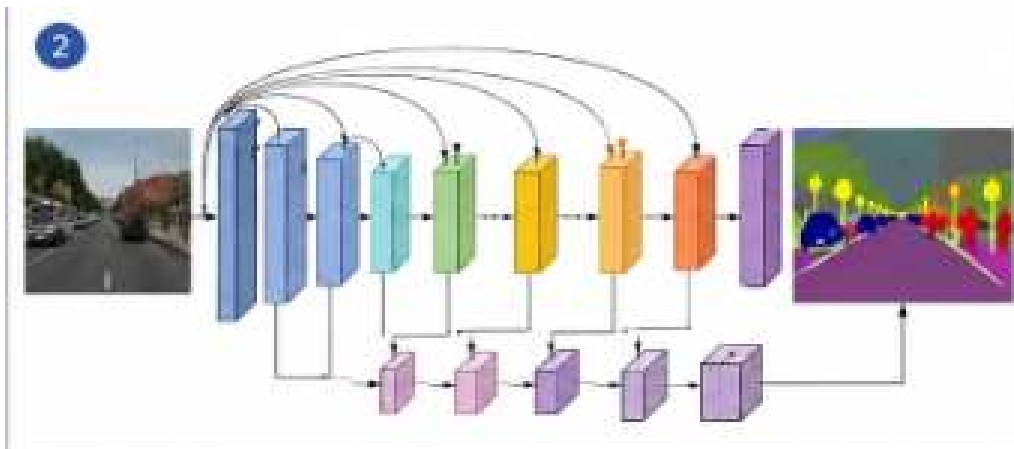
- Faster convergence
- Reduced training time
- Better generalization

- Reduced overfitting
 - Improved classification accuracy
 - Effective learning from limited labeled images
-

Q2. DenseNet and PixelNet for Road-Scene Segmentation

Proposed Architecture

A smart-city road-scene analysis system can combine **DenseNet** for **feature extraction** and **PixelNet-based pixel-wise prediction** for semantic segmentation.



Architecture

Input Road Image → DenseNet Backbone → Dense Feature Extraction → Feature Fusion → Upsampling → Pixel-wise Classifier → Segmentation Map

The system can classify every pixel into categories such as:

- Road
- Pedestrian
- Vehicle
- Traffic sign
- Building
- Sky
- Other objects

DenseNet Component

DenseNet uses dense connections between layers. Each layer receives the outputs of previous layers.

For example:

Layer 1 → Layer 2 → Layer 3

Instead of only passing the previous layer's output, DenseNet provides:

Layer 1 + Layer 2 → Layer 3

This allows previously learned features to be reused.

PixelNet Component

PixelNet focuses on **pixel-level classification**. Instead of predicting a single class for an entire image, it predicts a class for each individual pixel.

For example:

- Road pixels → Road
- Human pixels → Pedestrian
- Car pixels → Vehicle
- Sign pixels → Traffic Sign

Proposed Processing

1. Input a road-scene image.
2. Resize and normalize the image.
3. Pass the image through DenseNet.
4. Extract low-level and high-level features.
5. Use dense connectivity for feature reuse.
6. Combine useful feature maps.
7. Use an upsampling/decoder stage to restore spatial resolution.
8. Apply PixelNet-style pixel-wise classification.
9. Generate the final semantic segmentation map.

Feature Reuse

DenseNet improves feature reuse because each layer can access information from earlier layers.

This:

- Reduces redundant feature learning.
- Improves information propagation.
- Improves gradient flow.
- Helps preserve useful visual information.

Segmentation Accuracy

The combination of DenseNet and PixelNet improves segmentation accuracy because:

- DenseNet extracts rich visual features.

- Low-level features preserve object boundaries.
- High-level features provide semantic information.
- Pixel-wise classification identifies individual objects.
- Feature fusion helps distinguish similar objects.

Computational Efficiency

DenseNet can be parameter-efficient because features are reused rather than repeatedly learned.

The architecture can therefore provide a balance between:

Feature extraction + segmentation accuracy + computational efficiency

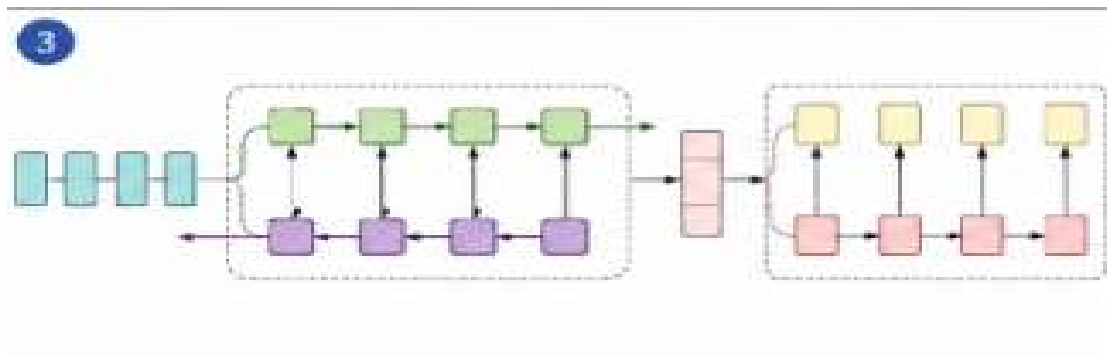
Overall Result

The proposed DenseNet–PixelNet system can accurately identify roads, pedestrians, vehicles, and traffic signs at the pixel level, making it useful for autonomous vehicles, traffic monitoring, and smart-city applications.

Q3. Bidirectional RNN and Encoder–Decoder for Multilingual Chatbot

Proposed Architecture

A multilingual chatbot can use a **Bidirectional RNN Encoder–Decoder sequence-to-sequence model** for translating customer queries while preserving contextual meaning.



Architecture

Input Sentence → Word Embedding → Bidirectional RNN Encoder → Context Representation → RNN Decoder → Translated Sentence

For example:

English:

"I want to cancel my order."

French:

"Je veux annuler ma commande."

Encoder

The encoder receives the input sentence as a sequence of words.

For example:

I → want → to → cancel → my → order

A Bidirectional RNN processes the sequence in two directions:

- Forward: beginning to end
- Backward: end to beginning

The forward RNN captures previous context, while the backward RNN captures future context.

Bidirectional Processing

For an input sequence:

$x_1, x_2, x_3, \dots, x_n$

the model calculates:

- Forward hidden states
- Backward hidden states

These states are combined to produce a richer representation of each word.

Decoder

The decoder receives the encoded representation and generates the translated sentence sequentially.

For example:

<START> → Je → veux → annuler → ma → commande → <END>

At every time step, the decoder uses:

- Previous output word
- Decoder hidden state
- Encoder context

to predict the next word.

Attention Mechanism

An attention mechanism can be added to improve translation.

It allows the decoder to focus on relevant input words when generating each output word.

For example, when generating:

"commande"

the model can focus strongly on:

"order"

from the original sentence.

Why Bidirectional RNN?

Bidirectional processing improves translation because the meaning of a word can depend on both previous and subsequent words.

For example:

"I saw a bank near the river."

The surrounding words help the model understand that "bank" refers to a river bank rather than a financial institution.

Advantages

- Better contextual understanding
- Improved word representation
- Better handling of ambiguous words
- Improved translation accuracy
- Better understanding of complete sentences
- Useful for multilingual customer-service applications

Overall Workflow

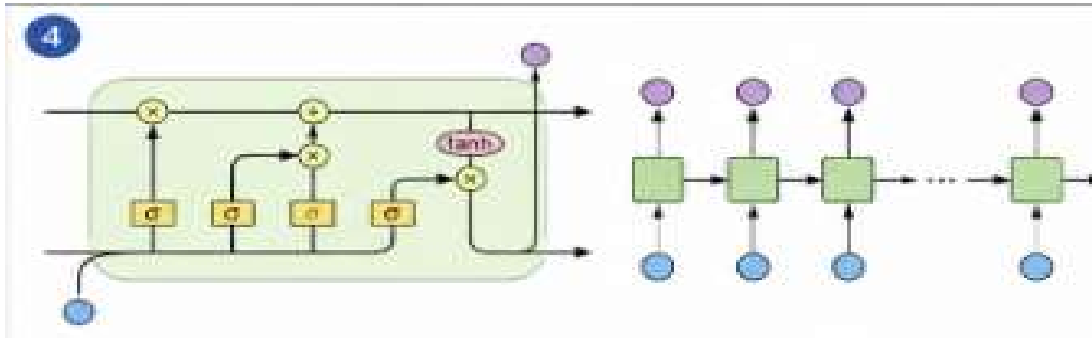
Customer Query → Tokenization → Embedding → Bidirectional Encoder → Context/Attention → Decoder → Translated Query

This architecture can provide context-aware multilingual translation for customer service chatbots.

Q4. BPTT and LSTM for Stock Price Prediction

Proposed System

A financial analytics company can use an **LSTM-based time-series prediction system** to forecast future stock prices from historical market data.



Input Data

The model can use:

- Opening price
- Closing price
- Highest price
- Lowest price
- Trading volume
- Moving averages
- Technical indicators

Architecture

Historical Market Data → Data Preprocessing → Sliding Window → LSTM → Dense Layer → Predicted Stock Price

For example, the model can use the previous 30 trading days to predict the next day's price.

LSTM Network

The LSTM contains three major gates:

1. Forget Gate

The forget gate determines which information from the previous cell state should be discarded.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

2. Input Gate

The input gate determines which new information should be stored.

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

3. Output Gate

The output gate determines what information should be passed to the next stage.

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

These gates allow LSTM to retain important information for longer periods.

Example Architecture

- Input layer
- LSTM layer with 64 units
- Dropout layer
- LSTM layer with 32 units
- Dense layer with 16 neurons
- Output layer with 1 neuron

The final neuron predicts the future stock price.

Backpropagation Through Time

BPTT is used to train recurrent neural networks by unfolding the network across multiple time steps.

For example:

Day 1 → Day 2 → Day 3 → ... → Day 30

The model produces a prediction after processing the sequence.

The difference between the predicted value and actual value is calculated using a loss function such as Mean Squared Error.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

The error is propagated backward through the time steps.

BPTT Training Steps

1. Provide historical stock data.
2. Divide data into time sequences.
3. Feed the sequence into the LSTM.
4. Generate the predicted price.
5. Calculate prediction loss.
6. Propagate the error backward through time.
7. Calculate gradients.
8. Update network weights.
9. Repeat for several training epochs.

Why LSTM Instead of Traditional RNN?

Traditional RNNs can have difficulty remembering information over long sequences because of the vanishing-gradient problem.

LSTM solves this problem using its memory cell and gates.

Traditional RNN	LSTM
Simple recurrent structure	Gated memory structure
Limited long-term memory	Better long-term memory
More affected by vanishing gradients	Reduces vanishing-gradient effects
Difficult for long sequences	Suitable for long sequences
Less effective for long-term dependencies	Better at learning temporal dependencies

Advantages

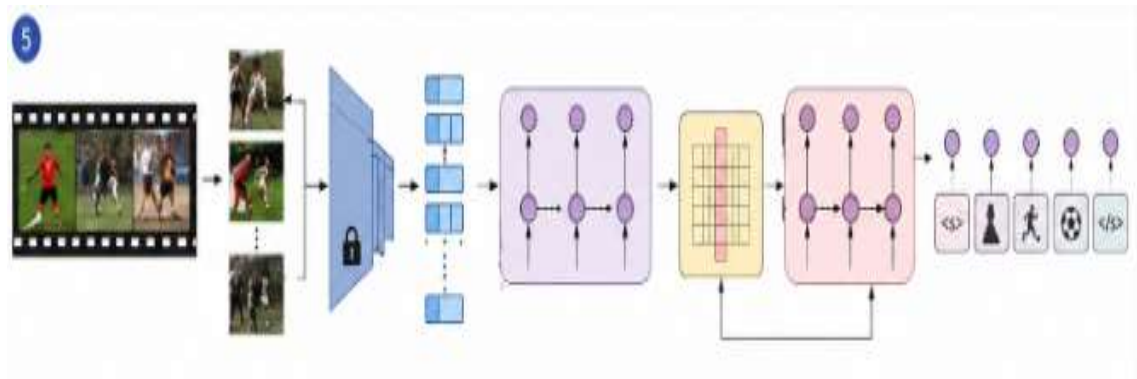
- Learns historical patterns.
- Handles sequential financial data.
- Retains important long-term information.
- Reduces the vanishing-gradient problem.
- Can model nonlinear relationships.

Therefore, LSTM is more suitable than a traditional RNN for stock-price time-series modeling. However, stock markets are highly uncertain, so the output should be treated as a forecast rather than a guaranteed prediction.

Q5. Integrated Deep Learning Solution for Automatic Video Captioning

Proposed Architecture

An automatic video-captioning system can combine **Transfer Learning for visual feature extraction** with an **LSTM Encoder–Decoder model** for generating natural-language captions.



Overall Architecture

Video → **Frame Extraction** → **Pre-trained CNN** → **Visual Features** → **LSTM Encoder** → **Context Representation** → **LSTM Decoder** → **Caption**

Component 1: Video Processing

The uploaded video is divided into a sequence of frames.

For example:

Video → **Frame 1** → **Frame 2** → **Frame 3** → ... → **Frame N**

Only selected frames can be processed to reduce computational requirements.

Component 2: Transfer Learning

A pre-trained CNN such as **ResNet50** or **InceptionV3** is used as a feature extractor.

The original classification layer is removed.

Each video frame is passed through the CNN to obtain a feature vector.

For example:

Frame 1 → **Feature 1**

Frame 2 → **Feature 2**

Frame 3 → **Feature 3**

The feature vectors represent visual information about objects, people, scenes, and activities.

Why Transfer Learning?

Transfer learning is useful because:

- The CNN has already learned powerful visual features.
- A large image dataset is not required for initial feature learning.

- Training time is reduced.
- It reduces the amount of data needed.
- It improves visual feature extraction.

Component 3: LSTM Encoder

The sequence of CNN feature vectors is given to an LSTM encoder.

The encoder learns temporal information between frames.

For example:

Frame 1: Person holding a ball

Frame 2: Person moving arm

Frame 3: Person throwing ball

The LSTM learns that these frames represent a continuous action.

Component 4: LSTM Decoder

The decoder converts the encoded visual information into a natural-language sentence.

For example:

<START>

↓

A

↓

person

↓

is

↓

throwing

↓

a

↓

ball

↓

<END>

The decoder predicts one word at a time.

Training Process

Training requires video-caption pairs.

For example:

Input: Video of a child playing in a park.

Target caption: "A child is playing in the park."

The model generates a caption and compares it with the target caption.

The error is calculated and propagated backward through the LSTM and other trainable components.

Attention Mechanism

An attention mechanism can be added between the encoder and decoder.

It allows the decoder to focus on important frames when generating particular words.

For example:

- When generating "**child**", focus on frames containing the child.
- When generating "**playing**", focus on frames showing the activity.
- When generating "**park**", focus on frames showing the environment.

Complete Workflow

1. Upload video.
2. Extract important frames.
3. Preprocess the frames.
4. Pass frames through the pre-trained CNN.
5. Extract visual feature vectors.
6. Send feature sequences to the LSTM encoder.
7. Encode temporal information.
8. Pass encoded information to the LSTM decoder.
9. Generate words sequentially.
10. Stop when the <END> token is generated.
11. Display the final caption.

Advantages

Transfer Learning

- Provides powerful visual features.

- Reduces training requirements.

CNN

- Identifies objects, scenes, and visual patterns.

LSTM Encoder

- Learns relationships between consecutive video frames.
- Captures temporal information.

LSTM Decoder

- Converts visual information into natural language.
- Generates captions sequentially.

Attention

- Helps the decoder focus on relevant frames.

Final Result

The integrated system combines spatial and temporal understanding:

CNN/Transfer Learning → What is present in the video

LSTM Encoder → What happens over time

LSTM Decoder → How to describe it in words